

Trellis

Property-Rich Mathematics with Proof-Explicit Elaboration

A proposal grounded in ProveIt, the Fermat’s Last Theorem formalization, Leant, and both round-two syntheses

Prepared for Vladimir Reshetnikov

6 September 2026

Abstract. A mathematician rarely repeats that a positive quantity is nonzero, that a chosen smooth function is continuous, or that a map preserves the structure for which it was introduced. A proof assistant must nevertheless justify every such use. The proposal here is not to relax that requirement, but to make these properties a second, proof-bearing layer of typing. Trellis keeps an object’s underlying Lean term fixed while accumulating scoped, checked refinements about it; separates selection of mathematical structures from inference of propositions about those structures; and gives functions behavioral contracts whose quantifiers, domains, and witnesses survive elaboration.

The recommended implementation is a conservative extension of Lean’s elaborator, not an extension of its kernel. A finite, demand-directed qualifier engine supplies routine property evidence. Existing Lean tactics, a certificate-producing computer-algebra interface, and Leant supply harder evidence under the same acceptance rule. Formal rules below establish a relative translation theorem, a restricted coherence result, termination and relative completeness of finite qualifier closure, and soundness of a small affine-certificate format. Worked examples cover sharp Fabius/Rvachev Lipschitz constants, stationary-phase differentiation, exact transport of Frey-curve coefficients, and behavioral synthesis beyond type inhabitation.

The article distinguishes what the previous round already established from the proposed third-round additions. An accompanying Python reference implementation passes 31 exact-arithmetic and protocol tests. It is not a Lean implementation, no Lean kernel was run for this article, and no measured proof-compression or usability claim is made.

Central recommendation

Make mathematical properties easy to request, infer, inspect, and reuse—while keeping every accepted refinement an ordinary proof about the exact object the author selected.

Contents

1	The design decision	3
2	What the second round settled—and what it did not	3
2.1	The inherited boundary is retained	4
2.2	The additions proposed here	4
2.3	An implementation criticism that remains binding	4
3	Source-grounded diagnosis	5
3.1	Scope and revision control	5
3.2	Fabius regularity: repeated consequences of stable properties	6
3.3	Stationary phase: data packages already do much of the work	6
3.4	Frey packages: properties versus generated environment management	6
3.5	Leant: several meanings of acceptance are already separated	7
4	What Lean already expresses	7
4.1	Four layers, not one new foundation	7
4.2	Related work and limits on novelty	8
5	A surface organized around objects and demands	8
5.1	Refinements without object replacement	8
5.2	A short proof can still have a visible mathematical plan	9
5.3	Explicit choices and two ways to introduce a witness	9
5.4	Partial work is permitted; partial acceptance is not	9
6	Core typing judgments and conservative elaboration	10
6.1	Logical state and an evidence index	10
6.2	The essential rules	10
6.3	Local refinements and exported packages	11
6.4	Statement holes versus proof holes	12
7	Predictable inference: a finite qualifier calculus	12
7.1	An open logic with a finite automatic fragment	12
7.2	Closure and its precise guarantee	12
7.3	What is and is not a principal refinement	13
7.4	Cycles, branches, and unknown properties	13
7.5	A useful frontier instead of a silent assumption	14
8	Select structures; infer properties	14
8.1	The boundary that proof irrelevance does not erase	14
8.2	A three-part classification of implicit information	14
8.3	Predictable elaboration across imports	15
8.4	Behavioral interfaces for representation maps	15
9	Behavioral types and their proof rules	15
9.1	An ordinary function type is often too weak a synthesis target	15
9.2	Regularity is behavioral information, not a formula annotation	16
9.3	Higher-order calls and recursive definitions	17
9.4	Effects: logical requirements are not execution outcomes	17
10	Worked example I: sharp Fabius and Rvachev regularity	18
10.1	The exact function-level contract	18
10.2	Where inference replaces repeated source work	18
10.3	Endpoint bounds and the domain that must stay visible	19
10.4	A stronger reusable interface than a one-off tactic	19
11	Worked example II: stationary phase and vanishing errors	19
11.1	Derivatives of a structured remainder	19
11.2	A useful generalization of the small-error helper	20

11.3 What should not disappear	20
12 Worked example III: Frey-package reasoning and model transport	21
12.1 A relational contract hidden inside a familiar proof	21
12.2 The package supplies guards, not stronger ambient mathematics	21
12.3 Integer division and rational division have different semantics	21
12.4 Keep an already short endgame short	22
13 A hybrid CAS with an exact, small first checker	22
13.1 The first certificate should match an actual demand	22
13.2 The reification bridge is part of the theorem application	23
13.3 Certificates for the worked bounds	24
13.4 General symbolic computation fits the same boundary	24
13.5 A verified algorithm is not a trusted execution channel	24
14 Leant integration: synthesize behavior, not just inhabitants	24
14.1 The contract is supplied before the candidate is accepted	24
14.2 Length contracts are useful, but not enough for every request	25
14.3 Where structural synthesis could genuinely help these examples	25
14.4 Exact edits and retained authority	26
15 Should Lean’s kernel type system change?	26
15.1 The recommended answer: not for the first implementation	26
15.2 A narrowly stated kernel research option	26
15.3 Proof irrelevance is not mathematical extensionality for free	27
16 A document view that cannot overstate its evidence	27
16.1 One semantic source for code and mathematical presentation	27
16.2 A stated reason is a constraint, not decorative prose	28
16.3 Editing, cases, and reuse	28
17 Implementation plan and the evidence actually supplied	28
17.1 A minimal vertical implementation, not a maximal language	28
17.2 What the companion implements	29
17.3 Executed checks and their limits	29
17.4 Additional paired tests for a Lean implementation	29
18 Evaluation and stopping rules	30
18.1 Treatments that isolate the source of a gain	30
18.2 Metrics must include mathematics and maintenance	30
18.3 Development examples are not held out	31
18.4 When to stop adding language	31
19 Answers to the third-round design questions	31
20 Conclusion	32
A A minimal grammar and export discipline	33
B Reproducing the companion checks	33
C Source register and provenance	34

1 The design decision

The central proposal is a language in which the following is an ordinary declaration, rather than an invitation to an unrestricted theorem prover:

```
variable f, d : Real -> Real
  with differentiable everywhere
  and derivative = d everywhere
  and range(d) contained in [-2, 2]

claim f is Lipschitz with constant 2
  by derivative_bound
```

The keywords are proposed syntax, not commands available in Lean today. Their meaning is precise: the declaration introduces a function, a specified derivative function, and proofs of quantified propositions about them. The claim demands another quantified proposition. The language does not decide that a displayed formula is differentiable merely because it looks familiar. Nor does it confuse an upper Lipschitz bound with the least Lipschitz constant.

The useful extension is a *property layer over ordinary dependent typing*. In a local proof, a real number can acquire evidence of positivity, integrality, a bound, or nonvanishing without becoming a different mathematical object each time. A function can acquire continuity, a derivative law, a range restriction, or a preservation theorem without being reconstructed as a tower of incompatible wrappers. At module boundaries, where evidence must travel with data, ordinary Lean structures and subtypes remain appropriate.

Three decisions distinguish the proposal.

First, structure selection precedes property inference. Selecting a multiplication, a topology, a scalar action, or an interpretation of division can change a statement. Proving that a fixed element is nonzero does not. These activities must not share one undifferentiated implicit-search mechanism.

Second, behavioral contracts are first-class demands. The useful type of a proposed function is often not merely $A \rightarrow B$, but that type together with a theorem about how inputs and outputs are related. A length-preserving list transformer need not preserve its elements. A denominator-clearing transformation need not preserve a domain. A function called an embedding need not reflect the relation a proof actually uses.

Third, automation is finite and proof-explicit where it claims predictability. A declared vocabulary of grounded property obligations admits a terminating closure algorithm. Arbitrary mathematical properties remain expressible, but automatic inference is not promised complete for them. Search beyond the finite fragment is an explicit provider operation; acceptance still means a Lean proof at the frozen target.

This changes what authors routinely specify, not what counts as evidence. The intended reduction in verbosity comes from reusable mathematical contracts, automatic instantiation of their routine consequences, and an interface that displays the mathematical argument instead of the elaborator's bookkeeping. Whether this is better than well-engineered ordinary Lean is an empirical question, not a conclusion of the design.

2 What the second round settled—and what it did not

Both requested syntheses were read, including their reciprocal corrections, implementation criticisms, and third-round questions [1, 2]. Their convergence is substantive, but much of it is

inherited from a shared brief. It is not evidence that nine independently implemented systems worked, and their feature counts are not usability measurements.

2.1 The inherited boundary is retained

The shared acceptance interface fixes an input i , an output family $O(i)$, and a relation $\text{Spec}(i, o)$. An accepted answer has the form

$$o : O(i), \quad p : \text{Spec}(i, o).$$

A computation may establish a conditional equality, a finite jet, a root witness, a complete solution set, or an enclosure. These are different propositions. Promotion between them requires a theorem with all its premises supplied. A provider’s failure is not a proof of negation. Accepted evidence must remain bound to its exact objects, environment, context, and request [1, 2].

Three corrections in the syntheses deserve particular emphasis. A complete solution set needs both soundness and coverage, not a witness plus coverage. Derivative transfer needs agreement on an appropriate neighborhood, not equality at one point plus an unrelated openness fact. An enclosure can establish equality when its bounds coincide, so a blanket prohibition of enclosure-to-equality is also wrong. Trellis inherits the corrected propositions rather than introducing an informal hierarchy of confidence levels.

The reports also distinguish the correctness theorem of an algorithm from evidence that a particular execution produced a correct answer. This distinction remains mandatory. A trusted-looking function name, a native execution, a verified callback receipt, or a source hash does not replace a proof at the original target.

2.2 The additions proposed here

Locus already proposed role-indexed guard closure; Concord already emphasized demands; Meridian already organized properties by proved implications; Vantage already separated observations from exact presentations. These precedents are credited through the detailed syntheses [1, 2]. Trellis does not rename them and count them as discoveries. Its main addition is a disciplined connection between such evidence and the *typing interface*, especially where structure choice and higher-order behavior would otherwise be hidden in elaboration.

2.3 An implementation criticism that remains binding

The syntheses argue that the next useful artifact should be a proved checker and a fair comparison, not merely another large grammar. That criticism applies to this proposal too. The supplied executable specification is deliberately small. It tests an exact certificate format and origin discipline, but it does not satisfy the requested future milestone of an end-to-end kernel-checked Lean implementation. The mathematical proofs in this article and the Python tests are different forms of evidence; neither is presented as that missing implementation.

A second tension in *Nine Workbenches* is worth resolving explicitly. Its questions mention Bell and Abel files as possible held-out tests, while its fourth-file discussion correctly explains that inspected design examples cannot remain held out. The latter rule governs the evaluation proposed here. Every file discussed below is development material, not test-set evidence.

Area	Already established in round two	Third-round proposal in this article
Refinements	Scoped guards and relation-specific results	A separate property-typing judgment over unchanged Lean terms, with explicit binder and export translations
Inference	Bounded guard closure, demand-driven observations	A finite grounded qualifier fragment with a closure theorem, proof DAGs, and a precise limit on any “principal refinement” claim
Structures	Statement seals and explicit conventions	Lexically selected structure contexts, distinct from proof-only refinement synthesis
Behavior	Exact specifications and finite-test cautions	Behavioral arrow rules, relational preservation/reflection, and proof-backed provider-law registration
Examples	Repeated binomial, Abel, and derivative examples	Fabius sharp regularity, stationary-phase data, and Frey-package transport as additional development cases
Computation	Proved checker and reification bridge required	A fully specified affine-bound certificate, its mathematical soundness proof, and an executable reference checker
Evaluation	Matched Lean baselines; no invented savings	Separate ablations for structure selection, qualifier inference, behavioral synthesis, and surface syntax

Table 1: An allocation of inheritance and proposed contributions, not a claim that refinement typing or proof-carrying computation is historically new.

3 Source-grounded diagnosis

3.1 Scope and revision control

The review used the following repository revisions, retrieved on 6 September 2026. Full identifiers and file paths are retained in the source register accompanying the article.

Repository	Revision prefix	Material directly inspected
Brainstorm	b052ec011beb	Both round-two TeX syntheses, including appendices and reciprocal corrections
ProveIt	24ce8bd743ea	<code>Regularity.lean</code> ; relevant portions of <code>SP1Taylor.lean</code> and <code>SP2Parts.lean</code>
Fermat’s Last Theorem	aa2d8b34692b	Frey-package definitions and proof; final Frey contradiction; README and proof-path description
Leant	3a40904be8a4	Verification receipts, behavioral-selection interface, and finite-spine Length contracts

No full repository build was performed. In particular, the FLT repository’s statements about its large build, comparator checks, and independent-kernel checks are the repository’s reports, not new verification results obtained here [9]. The purpose of inspecting it is to identify language-design pressures in actual source.

3.2 Fabius regularity: repeated consequences of stable properties

The ProveIt file `Regularity.lean` proves global Lipschitz estimates for the bounded Fabius function and Rvachev’s up-function, proves that the constant 2 is optimal, and derives endpoint majorants [3]. Its first estimate follows a recognizable mathematical argument: the derivative is $2u(2x - 1)$ and $0 \leq u \leq 1$, hence its absolute value is at most 2; the derivative-bound theorem gives the Lipschitz estimate.

The Lean source explicitly obtains the two range inequalities at the relevant argument, rewrites the derivative, handles the absolute value, solves a linear inequality, and converts a real norm inequality to the nonnegative-real norm interface. For the up-function it repeats the pattern with four range inequalities and two signs. None of these facts is expendable. But most are consequences of the same function-level contract, instantiated at different terms.

The design opportunity is not a magic command that knows the Fabius function. It is a reusable path

$$\text{range law} + \text{derivative law} \longrightarrow \text{derivative bound} \longrightarrow \text{Lipschitz bound},$$

with all objects and quantifiers retained. Sharpness requires a second path from a derivative value attaining the bound. A language that hides that distinction has made the proof shorter by losing information.

3.3 Stationary phase: data packages already do much of the work

The stationary-phase development introduces a `Data` structure with $g \in C^4$, interval endpoints, positive parameters, normalization at zero, and quantified derivative bounds. It derives continuity and derivative facts for named successive derivatives and remainder functions [4]. This is already a property-rich mathematical object in ordinary Lean.

The next file differentiates

$$w(x) = \frac{r_1(x)}{xg_1(x)}$$

under nonvanishing assumptions, proves bounds, and closes estimates containing a small ε -error [5]. The exposed work includes retrieving fields, composing derivative laws, proving the denominator nonzero, normalizing the quotient-rule result, and transporting inequalities through multiplication and division. Trellis should automate the repeated consequences, not claim to have invented the data package or its analytic theorems.

3.4 Frey packages: properties versus generated environment management

The inspected Frey package stores nonzero integers a, b, c , a prime $p \geq 5$, the Fermat equation, a coprimality condition, and residue-class normalizations. Its derived lemmas obtain positivity and oddness of p , pairwise coprimality consequences, nonvanishing of a product, and divisibility information. These are natural refinement-inference examples [6].

A different phenomenon occurs in the file proving the final Frey contradiction. Its mathematical endgame is already a short composition: irreducibility and modularity produce a nonzero level-two cusp form; the level-two vanishing theorem contradicts its nonzeroness. The surrounding generated source contains long instance- and simplifier-management preambles [7]. It would be misleading to count removal of this pipeline scaffolding as compression of a long mathematical argument. The evidence instead motivates a separate experiment in explicit structure environments and reproducible search profiles.

The repository also warns that names and generated prose are not authoritative descriptions of the strength of a theorem. For example, its proof-path discussion distinguishes trace-congruence

formulations from representation isomorphisms and records unused formal hypotheses in a particular modularity-related step [8]. A typed narrative should preserve that precision; it cannot infer the content of “modularity” or “embedding” from the English label.

3.5 Leant: several meanings of acceptance are already separated

Leant’s **Verified** wrapper is explicitly a receipt that a supplied callback accepted an exact candidate. Its documentation says this is not behavioral evidence, a solver certificate, or a kernel proof [10]. The behavioral-selection interface seals a bounded partition of exact receipts in a fresh epoch; it explicitly does not establish the candidates’ behavior [11].

The Length contract interface is particularly relevant to this brief. It supplies exact spine identities, argument roles, candidate-case policies, and behavioral transfer laws. These laws are passive assertions supplied by an integration layer, not facts inferred from provider names or implementations. The handoff binds them to retained provenance without claiming to manufacture universal behavioral proofs [12]. This is a useful boundary, not a bug to be fixed by relabeling it “verified.” The proposed extension supplies a Lean theorem at precisely the point where stronger semantic authority is required.

4 What Lean already expresses

Lean already has dependent function types, propositions as types, structures with proof fields, subtypes, and extensible elaboration. A subtype $\{x : A \mid P(x)\}$ carries a value and a proof; its runtime representation can erase the proof wrapper. Different proofs of the same proposition are irrelevant to Lean’s logical equality of proofs [13, 14]. Consequently, “add types for positive reals” is not, by itself, a new type-system proposal.

Similarly, type classes can carry operations together with their laws, or be indexed by values and predicates. Instance synthesis is a form of type-directed term construction, with success, failure, and stuck states. It is not merely a lookup table [15, 16]. Local instances, sections, theorem parameters, and ordinary **have** declarations already provide much of the proposed workspace.

The practical gap is the following combination: proof-only properties often require manual extraction and transport; packaging the same function in multiple structures may complicate interoperability; automatic elaboration can silently select meaning-bearing structures; and synthesized inhabitants of a simple type may not satisfy the mathematical behavior an author had in mind. Trellis addresses this combination at the source-to-proof boundary.

4.1 Four layers, not one new foundation

Layer	Responsibility	Proposed change
Lean kernel	Check proof terms and types in an environment	None in the recommended design
Lean libraries	State definitions, analytic facts, algebraic laws, and checker soundness theorems	Add a small set of reusable contracts and proof-producing adapters
Property elaborator	Instantiate properties, assemble evidence, retain origins and scopes	New demand-directed service over ordinary Lean terms
Mathematical surface	Express declarations, goals, chosen reasons, and visible distinctions	Optional syntax and an evidence-backed document view

This is a genuine extension of the *source language’s typing discipline*, but not necessarily of Lean’s core type theory. That distinction answers the user’s type-system question directly. Stronger mathematical types are desirable; a larger trusted kernel is not their default implementation strategy.

4.2 Related work and limits on novelty

Refinement types have a substantial history. Liquid Types combine predicate abstraction with type inference, while F* combines dependent/refinement typing with automated proof support and effectful programming [21, 22]. Polymorphic-refinement synthesis explicitly uses specification decomposition to reduce the combinations a synthesizer must consider [23]. These are precedents for the behavioral interface proposed here, not evidence that it has already been implemented for Lean mathematics.

Lean’s current documentation also includes compositional verification-condition generation for monadic programs through `mvccgen`; its 2026 releases continue to evolve this machinery [19, 20]. It would therefore be incorrect to present pre/postcondition reasoning or effectful verification as an absent capability of Lean. Trellis focuses on mathematical object properties, exact observation kinds, and interaction with existing theorem and synthesis infrastructure.

There is also serious work on changing definitional equality and justifying dependent coercive subtyping, including definitional functor laws for container-like types [24]. That work motivates a narrowly scoped research option in Section 15; it does not justify treating arbitrary proved equalities as computational equalities in Lean.

5 A surface organized around objects and demands

5.1 Refinements without object replacement

The principal local declaration has the form

```
variable p : Nat with prime p and 5 <= p
variable x : Real with 0 < x
variable F : BoundedFabius with IsFabius F
```

In assumption position this introduces the same ordinary binders one would write explicitly in Lean: the data variable followed by named proof assumptions. In definition position it instead creates proof obligations:

```
let x : Real with 0 < x := expression
```

The latter is not allowed to assume its own positivity. The value is elaborated, its meaning-bearing parameters are fixed, and positivity must be proved before the declaration is accepted.

An explicit command can add a proved fact about an existing object:

```
refine x with nonzero by positivity_implies_nonzero
```

This adds evidence of $x \neq 0$; it does not replace x by a subtype object in every subsequent expression. The main implementation is an index of proof terms about the original Lean expression. A user needing a transportable package can request an ordinary subtype or structure explicitly.

A refinement is not restricted to a one-word adjective. It may be a quantified relation, such

as $\forall y, 0 \leq u(y) \leq 1$, or a property involving other anchored objects. Relations among several objects are represented by predicates with those objects as parameters, not by duplicating their identities in unrelated registries.

5.2 A short proof can still have a visible mathematical plan

For the Fabius example, the intended surface is:

```
context F : BoundedFabius with IsFabius F
let f := fabiusReal F
let u := rvachevUp F

claim Lipschitz(f, 2):
  for arbitrary x : Real:
    bound abs(derivative f at x) <= 2
      using derivative_law(f), range_law(u)
      by affine_bounds
    conclude by derivative_bound

claim least_Lipschitz_constant(f) = 2:
  upper from Lipschitz(f, 2)
  lower from derivative f at 1/2 = 2
```

The declared names in this display denote registered semantic roles with ordinary Lean theorem types. They do not authorize inference from English phrases. The last claim is shorthand for a least-element proposition about nonnegative Lipschitz constants, not a call to an unspecified optimization oracle.

The `for arbitrary` binder is essential. The elaborator introduces one arbitrary real x , proves the bound in that context, and abstracts over x . Sampling many real arguments is not an implementation of the same construct.

5.3 Explicit choices and two ways to introduce a witness

The surface distinguishes a fixed object from a construction problem:

```
variable Delta : FormalSeries R with F(Delta) = 0
construct J : Jet R N satisfying agrees_below N J Delta
```

The first declaration binds an arbitrary solution with its supplied law. The second delegates a value whose specification is fixed. A solver may choose J ; it may not replace Δ by its preferred root. Root selection, an ordering, a branch of an inverse function, and a representation route are data choices, not merely different proofs of one proposition.

Within a proof, ordinary existential elimination permits a local witness. Returning a computational witness outside that proof is a different operation. If the only existence evidence is in `Prop`, the implementation must use an appropriate constructive interface or explicitly permitted noncomputable choice. A proof that an inverse exists does not supply an executable inversion algorithm.

5.4 Partial work is permitted; partial acceptance is not

An unfinished declaration can contain a value and pending obligations. It must display its missing contract and cannot be imported as a theorem. A conditional theorem $G \rightarrow P$ is a complete

theorem of a different type; it may be published as conditional. These two states must never be conflated.

Every formal assertion in an accepted document must be checked, including an intermediate assertion unused by the final proof. Explanatory prose may be explicitly marked commentary, but a false formal assertion cannot become harmless merely because a later tactic found another route.

6 Core typing judgments and conservative elaboration

6.1 Logical state and an evidence index

Let $\mathcal{E}$ be a Lean environment and Γ an ordered dependent context. Let $\mathcal{S}$ record the selected structure terms used when elaborating the source. These structure terms themselves occur in $\mathcal{E}$ or Γ ; $\mathcal{S}$ has no additional logical authority. Let Δ be an index of previously checked proofs, with their source, object, and scope information. Again, Δ is an index, not a second collection of axioms.

The complete refinement judgment is written

$$\mathcal{E}; \mathcal{S}; \Gamma; \Delta \vdash e : A \triangleright P \rightsquigarrow p.$$

It means that e has already been elaborated at type A and the output is a Lean term $p : P(e)$ in the exact context. For a row $\langle P_1, \dots, P_k \rangle$, the output is a tuple of proofs, equivalently a proof of their finite conjunction. The empty row imposes no extra condition.

An implementation also needs incomplete states, but they are not complete derivations of this judgment. Pending metavariables live in a draft elaboration state and must be solved or bound by an explicitly conditional statement before export.

6.2 The essential rules

Introduction. If ordinary Lean checking establishes $e : A$ and $p : P(e)$, attach the property without changing e .

Projection and conjunction. From proofs of $P(e)$ and $Q(e)$, build a proof of $(P \wedge Q)(e)$; conversely project either component. These are ordinary conjunction operations.

Subsumption by evidence. If $p : P(e)$ and

$$h : \forall z : A, P(z) \rightarrow Q(z),$$

then the new evidence is $h \circ p : Q(e)$. No semantic subtype relation is decided by fiat. The implication theorem may have additional parameters, but all of them must have been instantiated and justified.

Application. For $f : A \rightarrow B$, a precondition $P : A \rightarrow \mathbf{Prop}$, and a postcondition $Q : A \rightarrow B \rightarrow \mathbf{Prop}$, define

$$\text{Beh}(f; P, Q) := \forall x : A, P(x) \rightarrow Q(x, f(x)).$$

Given $h_f : \text{Beh}(f; P, Q)$ and $h_x : P(x)$, attach the proof $h_f x h_x$ to the application $f(x)$. The application is not changed in order to make the postcondition easy.

Equality transport. If $h : e = e'$ and $p : P(e)$, use Lean's equality eliminator to obtain $P(e')$. Merely having similar pretty-printed terms, the same polynomial, or isomorphic ambient objects is not this rule.

Implication introduction. A branch with local assumption $g : G$ may prove $p : P$. Closing it produces $\lambda g.p : G \rightarrow P$. It does not produce P in the parent context.

Universal introduction. A proof of $P(x)$ for a fresh arbitrary $x : A$, with the usual scope restrictions, yields $\forall x : A, P(x)$. It does not follow from facts proved at finitely many selected values.

Certified-provider rule. A producer may return a candidate term or certificate. The receiver checks a proof of the exact demanded proposition, or applies a registered checker-soundness theorem after checking execution and reification. A provider’s status string is never a premise of this rule.

6.3 Local refinements and exported packages

An assumption declaration

$$x : A \text{ with } P(x), Q(x)$$

can elaborate to the telescope $(x : A)(h_P : P(x))(h_Q : Q(x))$. A local refinement adds a proof binding to that telescope. An exported value-with-evidence can elaborate to

$$\{x : A \mid P(x) \wedge Q(x)\}.$$

These translations serve different interfaces. The local form avoids repeated coercions and preserves the raw object. The exported form makes evidence available to a client that did not participate in the original elaboration.

Rows contain proof fields. A row whose later facts depend on a newly chosen data witness must make that witness a separate binder. This restriction prevents an apparently harmless “extra property” from silently choosing a basis, an inverse, a root, or a representation.

Theorem 6.1 (Relative soundness of the complete core). *Assume that initial evidence terms typecheck in $\mathcal{E}; \Gamma$, registered deduction rules are Lean theorems, provider results are checked at their exact frozen types, and no unapproved axiom or unresolved obligation is admitted. Every finite complete derivation of the above refinement judgment translates to a Lean term $p : P(e)$ in that environment and context. No new axiom is required by the refinement rules themselves.*

Proof. Induct on the derivation. Introduction uses its checked premise. Projection and conjunction use the corresponding Lean constructors and eliminators. Subsumption and behavioral application use ordinary dependent application of the indicated theorem to the induction hypotheses. Equality transport uses equality elimination. Implication and universal introduction use lambda abstraction over the corresponding local binder; their scope conditions ensure the resulting term has no escaped free variable. A provider node contributes the checked exact-target term or the application of its checked soundness theorem to the checked execution and reification premises. Thus each rule constructs a term of the declared type from terms supplied by smaller derivations. The axiom dependencies of a constructed term come only from its environment and constituent evidence, which are subject to the assumed policy. $\square$

This is a theorem about the specified rules. It is not a mechanized theorem about a finished compiler. A compiler can still elaborate the wrong statement, and a kernel will faithfully prove that wrong statement. Consequently, the frozen elaborated statement and its mathematical rendering must be reviewed independently of the fact that the proof checks [18].

Proposition 6.2 (Restricted object stability and proof coherence). *For same-carrier, proof-only local refinements, adding another accepted property does not change the underlying term e . Two accepted proofs of a fixed property $P(e)$ give the same logical property evidence up to proof irrelevance. This does not identify different structures, different chosen witnesses, or different subtype predicates.*

Proof. The introduction, conjunction, and subsumption rules add proof terms but contain no operation replacing the data term e . Induction over a sequence of these refinements therefore leaves e unchanged. For a fixed proposition $P(e)$, proof irrelevance identifies its proofs. If one explicitly packages (e, p) and (e, q) in the same subtype, equality follows from equality of their value fields and proof irrelevance. This argument says nothing about subtypes defined by different predicates, or about a data-changing coercion introduced for a different carrier. $\square$

The restriction is important. Dependent transport, base change, quotient representatives, and maps between distinct carriers can change terms. They require their own checked equations or relational contracts; they are not covered by a slogan that all refinements erase to the same thing.

6.4 Statement holes versus proof holes

A statement may contain inference variables while its syntax is first elaborated. But before proof search, all meaning-bearing choices must be fixed: carrier types, quantified data, operations, domains, and intended result kinds. A proof hole can then ask for evidence of that fixed proposition. A declared construction hole can ask for a value satisfying a fixed specification. Search must not fill an unresolved theorem parameter with a convenient constant and present the resulting narrower statement as the original theorem.

This is not a ban on implicit arguments. It is a staging rule: ordinary elaboration resolves them, the result is sealed, and evidence discovery may not alter that seal.

7 Predictable inference: a finite qualifier calculus

7.1 An open logic with a finite automatic fragment

Arbitrary Lean predicates can appear in refinements. Nevertheless, the predictable automatic layer uses a finite set $\mathcal{Q}$ of well-typed, *grounded* propositions generated for the current demand. Grounded means that the objects, structures, and relevant data arguments are already fixed; it does not mean that those objects are numerical constants.

For example, after introducing an arbitrary $x : \mathbb{R}$ in the Fabius proof, the expressions $u(2x - 1)$ and $2u(2x - 1)$ are fixed terms in that local context. The qualifier set can contain their lower bounds, upper bounds, and the derivative identity at x . The proof later abstracts over x . The finite engine has not enumerated real inputs.

Each registered ground rule has the form

$$P_1 \wedge \cdots \wedge P_k \longrightarrow Q, \quad P_i, Q \in \mathcal{Q},$$

with a Lean theorem instance supplying the implication. Rules can express positivity-to-nonzero, a product's nonvanishing, a derivative-law projection, or a particular range-law instantiation. Instantiation of general theorems belongs to an earlier bounded rule-generation step, not to an unbounded saturation over all possible terms.

7.2 Closure and its precise guarantee

Let $S_0 \subseteq \mathcal{Q}$ be the initially available proved facts. Define

$$T(S) = S \cup \{Q : \text{some registered rule for } Q \text{ has all its premises in } S\}.$$

Iterate $S_{n+1} = T(S_n)$ until no new fact is added. Whenever a conclusion is added, retain the theorem instance and references to already available premise proofs.

Theorem 7.1 (Finite qualifier closure). *For a finite qualifier universe and finite ground-rule set:*

1. *closure terminates after at most $|\mathcal{Q} \setminus S_0|$ strict fact additions;*
2. *every produced fact has a finite proof DAG assembled from initial evidence and registered theorem instances;*
3. *the final fact set is the least rule-closed superset of S_0 ;*
4. *a fair rule schedule computes the same final fact set, although its proof DAGs can differ.*

Proof. The sequence is monotone and can add only members of the finite set $\mathcal{Q}$, proving termination. At each strict addition, all premise proofs already exist; applying the rule theorem constructs the new proof and edges to earlier nodes. Thus the dependency graph is acyclic even if the rule graph has cycles. Any rule-closed superset of S_0 contains S_1 and, inductively, every S_n , hence contains the fixed point. Conversely, the fixed point is rule-closed by construction. Fair scheduling eventually applies every enabled rule, so it reaches precisely this same least closed set. $\square$

With premise counters and reverse incidence lists, the bookkeeping can be linear in the number of ground facts plus ground-rule premise occurrences, once each rule is processed when enabled. This bound excludes rule generation, theorem instantiation, Lean typechecking, and proof-term size. It is not a performance claim about the complete elaborator.

7.3 What is and is not a principal refinement

Within this fixed vocabulary, closure gives all consequences obtainable from the supplied rules. It can therefore provide a canonical *set of available qualifiers*. It is not a strongest mathematical type among all Lean predicates. Changing the qualifier vocabulary, adding a theorem, or allowing a new representation can expose more properties. There is no promised principal refinement in unrestricted dependent mathematics.

For demand-directed execution, form the backward dependency subgraph of the demanded qualifiers and include every premise of every retained ground rule. Saturating this closed subgraph is complete relative to those retained rules. Pruning it heuristically can be useful, but forfeits that relative-completeness claim. A bounded search that fails remains inconclusive.

7.4 Cycles, branches, and unknown properties

Rules $P \rightarrow Q$ and $Q \rightarrow P$ do not create either fact from nothing. A conditional cancellation theorem cannot use its own conclusion to manufacture the denominator guard that would justify it. A proof DAG permits cycles only through a separately justified induction or recursive theorem, not by a circular reference in an evidence cache.

All qualifier keys include the actual elaborated proposition and its context. A branch-local proof of $x \neq 0$ cannot be retrieved in its sibling merely because the variable is printed as x in both branches. Parent-to-child weakening is sound when the context embedding is valid. More general reuse needs a checked substitution and evidence for the old assumptions; a hash comparison is not that proof.

Failure to derive P means only that no evidence of P was produced by this procedure. It does not even mean that P is false in the intended mathematics. The executable companion deliberately includes a true positive constant inequality whose proposed certificate is unsupported, to make the distinction between a rejected certificate and a refuted proposition explicit.

7.5 A useful frontier instead of a silent assumption

When a demanded property remains open, the interface should expose one or more missing-premise frontiers, such as:

```
Cannot establish the quotient derivative at x.
Missing: g1(x) != 0.
Available route:
  abs(g1(x)) >= lambda2 * abs(x)
  lambda2 > 0
  x != 0
The first bound is available only when x belongs to [a,b].
```

This is a demand explanation, not a new hypothesis in the theorem. Selecting a route that introduces an interval premise must either use an already proved membership fact, open a visible conditional subgoal, or change the proposed statement explicitly.

8 Select structures; infer properties

8.1 The boundary that proof irrelevance does not erase

An additive and a multiplicative monoid can live on closely related carriers. Different scalar actions can make the same underlying function linear in one setting and not another. Different topologies can change continuity. These are not differences between proofs of one fixed proposition. They change the operations or predicates in the proposition [15].

Trellis therefore proposes a *structure context*: a lexical selection of the data-bearing dictionaries that interpret a mathematical region. Its name is a human handle; its contents are ordinary Lean terms. A schematic example is:

```
structure_context algebraic:
  carrier R
  ring_operations rho
  rational_algebra iota
  module_action on M := mu

within algebraic:
  variable a : R with IsUnit a
  claim multiplication_by(a) is invertible
```

This does not require a new universe of structures. The implementation can elaborate the selected dictionaries as explicit parameters and local instances. What is new is the policy that their identities are retained in the statement seal and cannot be changed by subsequent proof search. Properties may be inferred aggressively once the relevant operations are fixed. Inferring `Nonzero(a)` from `Positive(a)` in a selected ordered field is proof construction. Choosing an ordered-field structure on an insufficiently specified carrier is a different task and must not be disguised as the same inference.

8.2 A three-part classification of implicit information

A useful classification is *operations*, *laws*, and *witness data*. Operations include multiplication, order, topology, and scalar action. Laws include associativity, continuity, nonvanishing, and quantified derivative identities, with the operations already fixed. Witness data include a basis,

an isomorphism, a chosen inverse implementation, and a root selector.

The classification is about semantic role, not just whether a particular Lean declaration happens to be a class or a structure. Some classes are proof-only; some apparently convenient witnesses determine later computations. A declaration can contain both data and laws. The elaborator must inspect the actual type and dependencies rather than infer the role from the declaration’s spelling.

For example, `IsUnit(a)` is a proposition, whereas a selected element of the `units` type is data with an inverse operation. Moving from the first to an executable witness requires a construction, or explicit noncomputable choice. The source should not promise execution solely because a mathematical existence fact was inferred.

8.3 Predictable elaboration across imports

The proposed policy has three stages. First, elaborate a statement and record the relevant selected dictionaries and coercion routes. Second, search for proof evidence without changing them. Third, upon a library or import change, re-elaborate and compare meaning-bearing terms before reusing evidence. If a formerly selected instance disappears, the system should report a migration obligation rather than silently choose another structure and keep an unchanged-looking statement.

This policy cannot instantly remove every instance problem in an existing development. Imported definitions may already be specialized to particular structures; their compatibility with the local selections still has to be checked. Likewise, the generated FLT preambles are not proof that ordinary Lean’s type-class system is inadequate. They identify a workload on which scoped instance selection and retained resolution plans should be compared against the current pipeline [7].

8.4 Behavioral interfaces for representation maps

Let $f : A \rightarrow B$ and let R_A, R_B be relations. Define

$$\begin{aligned} \text{Preserves}(f; R_A, R_B) &:= \forall x, y, R_A(x, y) \rightarrow R_B(f(x), f(y)), \\ \text{Reflects}(f; R_A, R_B) &:= \forall x, y, R_B(f(x), f(y)) \rightarrow R_A(x, y). \end{aligned}$$

These are separate behavioral properties. Preservation of polynomial identities by a ring homomorphism does not imply reflection of ideal membership. The injection $\mathbb{Z} \hookrightarrow \mathbb{Q}$ is a familiar counterexample: $X \in (2X)$ over $\mathbb{Q}[X]$, while not over $\mathbb{Z}[X]$ [1, 2]. The appropriate type-level interface names the relation being transported, not just a generic “safe view.”

Maps compose predictably when their relevant laws compose. If f and g preserve the corresponding relations, $g \circ f$ preserves them by two implication applications. If both reflect them, the composition reflects them in reverse order. No inverse map is needed for these statements; conversely, a proof of injectivity alone only supplies equality reflection, not reflection of every mathematical predicate.

9 Behavioral types and their proof rules

9.1 An ordinary function type is often too weak a synthesis target

A function of type $\text{List}(A) \rightarrow \text{List}(A)$ may return the empty list, reverse its input, duplicate it, or ignore its elements. Even a length-preserving function may replace every element with a fixed

value. A structural inhabitant can therefore be impeccably well typed and still be irrelevant to the intended mathematics.

For pure functions, the basic contract $\text{Beh}(f; P, Q)$ from Section 6 is enough to express input/output laws. Other function properties are genuinely relational or higher order: monotonicity compares two inputs; additivity compares three evaluations; continuity quantifies over neighborhoods; a derivative law mentions another function. They remain ordinary predicates $B(f)$, not necessarily reducible to a small built-in menu.

The refinement layer permits both forms:

```
f : (x : A where P(x)) -> (y : B where Q(x,y))
g : A -> B with monotone g and preserves_addition g
```

The first has a checked translation to a function and a quantified pre/postcondition theorem. The second attaches explicitly defined predicates. Neither form silently chooses an order or addition operation.

Proposition 9.1 (Behavioral weakening). *Suppose $\text{Beh}(f; P, Q)$, $\forall x, P'(x) \rightarrow P(x)$, and*

$$\forall x, y, P'(x) \rightarrow Q(x, y) \rightarrow Q'(x, y).$$

Then $\text{Beh}(f; P', Q')$.

Proof. Given x and $P'(x)$, derive $P(x)$, apply the original contract to obtain $Q(x, f(x))$, and apply the postcondition implication. Abstract over x and its precondition proof. $\square$

The direction matters. A function requiring less of its caller can serve a caller offering more. A result guaranteed to satisfy a stronger postcondition can serve a weaker demand. Automatically reversing either implication is an unsound form of behavioral subtyping.

Proposition 9.2 (Dependent composition). *Suppose $\text{Beh}(f; P, Q)$ and $\text{Beh}(g; S, T)$, together with a bridge*

$$b : \forall x, y, P(x) \rightarrow Q(x, y) \rightarrow S(y).$$

Then $h = g \circ f$ satisfies

$$\forall x, P(x) \rightarrow \exists y, y = f(x) \wedge Q(x, y) \wedge T(y, h(x)).$$

Proof. For x satisfying P , choose the witness $y = f(x)$. The first contract proves $Q(x, y)$. The bridge proves $S(y)$, so the second contract proves $T(y, g(y))$. Since $h(x) = g(f(x))$, these are exactly the required components. $\square$

Keeping the intermediate witness in the postcondition avoids losing dependencies. A specialized adapter may project the information a consumer needs, but the general contract does not commute existential and universal quantifiers or make a pointwise witness independent of its input.

9.2 Regularity is behavioral information, not a formula annotation

For real functions, define

$$\text{DerivOn}(f, d, U) := \forall x \in U, \text{HasDerivAt}(f, d(x), x).$$

This definition uses a full derivative at each listed point. A within-set derivative is a different predicate and receives a different role. Likewise,

$$\text{RangeOn}(f, U, V) := \forall x \in U, f(x) \in V$$

and a Lipschitz contract on U records its quantified pair of points and constant.

If f has derivative d on U , g has derivative e on V , and f maps U into V , the chain rule yields a derivative contract for $g \circ f$ on U . The chain rule is an existing analytic theorem; the new language work is to infer and retain the map-into condition rather than require authors to repeat it at every application.

If two functions agree on an open set U , a point $x \in U$ has a neighborhood on which they agree, and a derivative law transfers there. Pointwise equality $f(x) = g(x)$ cannot do this: $f(t) = t$ and $g(t) = 0$ agree at zero but have different derivatives. The qualifier vocabulary must distinguish these two propositions even when both are printed near the same point [1, 2].

9.3 Higher-order calls and recursive definitions

A combinator receiving a function argument should demand the behavioral property it uses. For example, a reindexing combinator needs bijectivity on specified finite index sets, not merely a function between their carriers. A limit/integral exchange needs the hypotheses of its selected analytic theorem, not a collection of plausible pointwise identities.

Recursive synthesis must also satisfy Lean’s termination or well-founded-recursion requirements, in addition to its postcondition. A proof of a length relation is not a proof of termination of an arbitrary implementation. The proposed language retains ordinary Lean recursion checking rather than inferring totality from a behavioral label.

9.4 Effects: logical requirements are not execution outcomes

For effectful programs, a pure predicate on $A \rightarrow B$ is insufficient. One needs a specified operational semantics, a weakest-precondition judgment, or an appropriate Hoare-style contract. Existing Lean verification-condition machinery is the natural host for those proofs [19, 20].

For the mathematical interface, it is useful to record three different kinds of information: required logical hypotheses; the observation being claimed, such as an enclosure or a jet; and operational state, such as a cancelled search or a stale edit. These must not be merged into one success bit. A complete proof under a hypothesis is still conditional. A cancelled search establishes no logical statement. A stale proof can remain valid for its old target while lacking authority to change the current document.

Ordinary mathematical evidence is reusable, not a consumable linear resource. Affine or linear tokens may be appropriate for committing an edit once, selecting an epoch, or owning a solver process. They are not a reason to forbid using the same positivity proof twice.

10 Worked example I: sharp Fabius and Rvachev regularity

10.1 The exact function-level contract

The source provides the following relevant facts about $f = \text{fabiusReal}(F)$ and $u = \text{rvachevUp}(F)$ for a specified F satisfying its Fabius laws [3]:

$$0 \leq u(t) \leq 1 \quad (t \in \mathbb{R}), \quad (1)$$

$$f'(x) = 2u(2x - 1), \quad u'(x) = 2(u(2x + 1) - u(2x - 1)), \quad (2)$$

$$u(0) = 1, \quad u(2) = 0. \quad (3)$$

Here the derivative equations come with derivative-existence proofs, not just equalities involving a totalized derivative operator. The contract also identifies the functions and their real domain; the symbol u is not free to change during normalization.

Theorem 10.1 (Sharp global Lipschitz constants). *Under the above contract, both f and u are 2-Lipschitz on $\mathbb{R}$, and every nonnegative Lipschitz constant for either function is at least 2.*

Proof. Put $a = u(2x - 1)$. By (1), $0 \leq a \leq 1$, so $0 \leq 2a \leq 2$ and hence $|f'(x)| \leq 2$.

For u , put $a = u(2x + 1)$ and $b = u(2x - 1)$. The required two inequalities have the explicit certificates

$$2 - 2(a - b) = 2(1 - a) + 2b \geq 0, \quad (4)$$

$$2 + 2(a - b) = 2a + 2(1 - b) \geq 0. \quad (5)$$

Thus $|u'(x)| \leq 2$. For distinct real x, y , the mean value theorem applied on the interval between them gives $|f(x) - f(y)| \leq 2|x - y|$ and the same inequality for u ; equal endpoints are immediate. At $x = 1/2$, (2) and (3) give $f'(1/2) = 2$ and $u'(1/2) = -2$. If a differentiable function h is K -Lipschitz, then for every nonzero real increment t ,

$$\left| \frac{h(x + t) - h(x)}{t} \right| \leq K.$$

Taking the derivative limit at x gives $|h'(x)| \leq K$, since the closed interval $[-K, K]$ contains every difference quotient. Applying this at $1/2$ to the two functions yields $2 \leq K$. Together with the upper bounds, this proves leastness. $\square$

This proof exposes the important distinction in behavioral types. The inferred property $\text{Lip}(f, 2)$ is an upper bound. A *sharpness witness*, consisting here of a derivative attaining magnitude 2, supplies the lower-bound argument. A normalizer may simplify the witness; it may not manufacture optimality from the upper bound.

10.2 Where inference replaces repeated source work

The qualifier engine instantiates the global range law at the two expressions needed by the derivative rule. The affine checker establishes the inequalities in (4)–(5). A theorem adapter turns the signed bounds into an absolute-value bound. Another adapter supplies the form expected by the Lipschitz theorem, including the nonnegative-real constant interface.

Every adapter has an ordinary theorem type. In particular, crossing from a real-valued inequality to a nonnegative-real norm inequality is not an untyped cast performed by a CAS. Its relationship to the exact constant and norm is checked. The proposed compact surface is useful only if its ordinary-Lean baseline is given these same adapters.

10.3 Endpoint bounds and the domain that must stay visible

The source also proves $f(0) = 0$ in the selected Fabius package and derives endpoint majorants [3]. From the Lipschitz bound,

$$|f(x)| \leq 2|x|.$$

For $x \geq 0$, this implies $f(x) \leq 2x$. It does not justify replacing $|x|$ by x for an arbitrary real input. A branch-sensitive property layer can remove that bookkeeping in the positive branch; it cannot remove the branch condition from the theorem.

A global formula such as $f(x) \leq 2 \max(x, 0)$ additionally uses the function's behavior on the nonpositive half-line. Trellis must show that this is a combination of a Lipschitz argument and a separate support/normalization law, not a purely algebraic weakening of the local estimate.

10.4 A stronger reusable interface than a one-off tactic

A general package can state the following theorem: if h is differentiable on $\mathbb{R}$, $|h'| \leq K$, and $|h'(c)| = K$, then K is its least nonnegative Lipschitz constant. This package is a reusable mathematical abstraction already assembled from familiar theorems. The property layer can discover that its premises are available. It need not contain special Fabius logic, and it must not expose a `least` field until the attainment premise has been proved.

11 Worked example II: stationary phase and vanishing errors

11.1 Derivatives of a structured remainder

The inspected source fixes $c = g''(0)$ and defines

$$r(x) = g(x) - cx^2/2, \quad r_1(x) = g'(x) - cx, \quad r_2(x) = g''(x) - c,$$

with derivative proofs linking these functions. It also defines

$$\Phi(x) = xr_2(x) - 2r_1(x).$$

These are definitions and laws of the same selected data package [4]. For readability write $g_1 = g'$ and $g_2 = g''$.

Proposition 11.1 (Guarded quotient differentiation). *At any real x with $x \neq 0$ and $g_1(x) \neq 0$, the function*

$$w(x) = \frac{r_1(x)}{xg_1(x)}$$

has derivative

$$w'(x) = \frac{\Phi(x)g_1(x) - r_1(x)(xr_2(x) - r_1(x))}{(xg_1(x))^2}. \quad (6)$$

Proof. The product rule gives derivative $g_1(x) + xg_2(x)$ for $xg_1(x)$. Its value is nonzero by the two guards. The quotient rule therefore gives numerator

$$xr_2(x)g_1(x) - r_1(x)(g_1(x) + xg_2(x)).$$

The numerator displayed in (6) expands to

$$xr_2g_1 - 2r_1g_1 - xr_1r_2 + r_1^2.$$

Subtracting the quotient-rule numerator leaves

$$r_1(-g_1 + x(g_2 - r_2) + r_1) = 0,$$

because $g_2 - r_2 = c$ and $r_1 = g_1 - cx$. Thus the two derivative values coincide. This is a polynomial identity in the named values after using the defining equations; the nonvanishing guards belong to the earlier analytic quotient rule. $\square$

The proof separates precisely the two services a hybrid proof assistant/CAS should combine. Analytic theorems establish existence and the quotient-rule expression. A certified algebraic normalizer proves equality of derivative values. The normalizer cannot replace the analytic theorem, and a correct numerator identity does not discharge the denominator guards.

The source's interval assumptions imply a bound $|g_1(x)| \geq \lambda_2|x|$ with $\lambda_2 > 0$ on the designated interval. At a nonzero point in that interval this implies $g_1(x) \neq 0$ [4, 5]. The qualifier engine can assemble this implication. Outside the interval, the global derivative theorem still accepts an independently supplied nonvanishing proof; it must not inherit the interval estimate without its membership premise.

11.2 A useful generalization of the small-error helper

The source contains an elementary helper proving $X \leq K$ from

$$X \leq K + c\varepsilon^2 \quad \text{for every } 0 < \varepsilon \leq \varepsilon_0, \quad c \geq 0, \quad \varepsilon_0 > 0,$$

using a carefully chosen square root and several inequalities [5]. Its mathematical pattern admits a broader contract.

Theorem 11.2 (Closing a vanishing-error estimate). *Let $e : (0, \varepsilon_0] \rightarrow \mathbb{R}$ tend to zero as $\varepsilon \downarrow 0$, where $\varepsilon_0 > 0$. If*

$$X \leq K + e(\varepsilon) \quad (0 < \varepsilon \leq \varepsilon_0),$$

then $X \leq K$.

Proof. Set $\varepsilon_n = \varepsilon_0/(n+1)$ for $n \geq 0$. These values lie in the specified interval and tend to zero from the right, so $e(\varepsilon_n) \rightarrow 0$. If $X > K$, let $\delta = (X - K)/2 > 0$. For sufficiently large n , $|e(\varepsilon_n)| < \delta$, whence

$$X \leq K + e(\varepsilon_n) < K + \delta = (K + X)/2 < X,$$

a contradiction. $\square$

In particular, $e(\varepsilon) = c\varepsilon^2$ satisfies this contract for *every* real c ; nonnegativity of c is not needed for this generalized theorem. This is a proposed reusable lemma, not a claim that the original source's signature was changed or mechanically improved here.

A behavioral type can record “tends to zero along this right-hand domain” and infer it compositionally for the displayed error term. The author writes a limiting argument once; the elaborator obtains the appropriate specialization. The filter or one-sided domain is part of the type. A finite numerical experiment near zero is not evidence for that convergence contract.

11.3 What should not disappear

The difficult analytic choices remain mathematical: the interval split, the treatment of the singular point, the derivative estimates, and the selection of a limit argument. Trellis aims to remove repeated propagation of their consequences. It should not hide the need for an integrability theorem or silently interchange limits and integrals merely because each component has a convenient property row.

12 Worked example III: Frey-package reasoning and model transport

12.1 A relational contract hidden inside a familiar proof

For a natural exponent $p > 0$, the map $z \mapsto z^p$ on integers preserves and reflects divisibility:

$$d^p \mid e^p \iff d \mid e. \quad (7)$$

This is the relation-specific behavior used by the Frey-package gcd proof [6]. It is stronger and more precise than saying that exponentiation is an embedding.

For completeness, preservation follows by raising a factorization to the p th power. For reflection, the case $e = 0$ is immediate; if $d = 0$, divisibility forces $e^p = 0$ and hence $e = 0$. In the remaining case, compare the exponents of each prime in the absolute values of d and e . Divisibility of the p th powers gives $p v_\ell(|d|) \leq p v_\ell(|e|)$ for every prime ℓ . Since $p > 0$, each inequality cancels to $v_\ell(|d|) \leq v_\ell(|e|)$, which is exactly divisibility of the integers up to their harmless unit signs.

The positivity condition cannot be omitted. At exponent zero, every integer’s zeroth power is 1, so divisibility of powers carries no information about the original integers. Nor can the behavior be weakened to “ $d \mid e^p$ implies $d \mid e$ ”: $4 \mid 2^2$ but $4 \nmid 2$.

Proposition 12.1 (Coprimalty transport). *If $p > 0$ and $a^p + b^p = c^p$ for integers a, b, c , then*

$$\gcd(a, b) = \gcd(a, c) = \gcd(b, c),$$

where gcds are nonnegative normalized gcds.

Proof. Let $d = \gcd(a, b)$. Since $d \mid a$ and $d \mid b$, d^p divides both a^p and b^p , hence divides their sum c^p . By (7), $d \mid c$, so $d \mid \gcd(a, c)$.

Conversely let $e = \gcd(a, c)$. Its p th power divides $c^p - a^p = b^p$, hence $e \mid b$ by the same reflection law. Thus $e \mid \gcd(a, b)$. Mutual divisibility of normalized nonnegative gcds gives equality. Interchanging a and b proves the remaining equality. $\square$

The source performs the same mathematical argument with explicit divisibility lemmas and conversions between integer and natural presentations of the gcd [6]. A Trellis version should expose the two divisibility directions and use the registered power-reflection behavior for the repeated conversions. It should not bury the proof in a new opaque “Frey tactic.”

12.2 The package supplies guards, not stronger ambient mathematics

The package’s assumption that p is prime and $5 \leq p$ supplies $p > 0$ and oddness. Its equation and gcd law then imply the other coprimality statements. Its nonzero fields give $abc \neq 0$ over the integers. These are natural local rule instances about fixed data.

None of them licenses replacing the integer ring by a field, changing the selected prime, or dropping the nonzero hypotheses from the package. An author can write the properties once in a declaration; a proof assistant must still retain their dependencies. A displayed “prime exponent” is not sufficient to infer oddness without excluding the prime 2.

12.3 Integer division and rational division have different semantics

The source defines both an integral and a rational Frey model, with the nontrivial coefficient expressions

$$a_2 = \frac{b^p - 1 - a^p}{4}, \quad a_4 = -\frac{a^p b^p}{16}.$$

The identical displayed notation does not make integer division the same operation as rational division [6]. A correct base-change argument needs divisibility proofs.

The package’s residue information provides them. Since $p \geq 5$ is prime, p is odd. The condition $a \equiv 3 \pmod{4}$ gives $a^p \equiv 3 \pmod{4}$. Since b is even and $p \geq 2$, $b^p \equiv 0 \pmod{4}$. Therefore

$$b^p - 1 - a^p \equiv 0 - 1 - 3 \equiv 0 \pmod{4}.$$

Also $b = 2k$ for some integer k , and $p \geq 4$, so $16 \mid b^p$ and hence $16 \mid a^p b^p$. Consequently both integer quotients are exact. Their images in $\mathbb{Q}$ equal the corresponding rational quotients. The remaining coefficients are constants, so their transport is immediate.

This example motivates a behavioral contract for the cast operation conditioned on exact divisibility. It is not a generic rewrite that commutes every integer quotient with the embedding into $\mathbb{Q}$. For instance, $(1 \text{ div } 4 : \mathbb{Z})$ maps to 0, not to $1/4$.

The proposed property layer could assemble the parity, power-divisibility, residue, and exact-quotient facts automatically. But the resulting proof still refers to the original coefficients and models. A statement that the resulting Weierstrass structures coincide under base change requires the relevant structure-extensionality theorem, not just similarity of their pretty-printed equations.

12.4 Keep an already short endgame short

The final contradiction in the inspected FLT file is already well represented by a short composition of its package theorems [7]. An optional mathematical display is:

```
assume P : FreyPackage
obtain f with nonzero f at level 2
  from level_lowering(P, modular(P), irreducible(P))
contradiction with all_level_2_weight_2_forms_vanish
```

This is an interface sketch whose semantic roles must be bound to the exact existing statements. It is not a fresh proof of Fermat’s Last Theorem. Its value is that the irreducibility, modularity, level, weight, and nonzeroness remain visible. It must not force authors to replace the shorter existing Lean term with a longer ritual.

The generated preambles should be studied separately as elaboration-environment overhead. A successful design might keep this final proof unchanged while improving the management of the structures and rules around it. That would be a useful outcome even if the source contains no new proof-language keywords.

13 A hybrid CAS with an exact, small first checker

13.1 The first certificate should match an actual demand

The Fabius/Rvachev bounds need linear combinations of known nonnegative quantities, not a general-purpose symbolic algebra system. This suggests a smaller first service than unrestricted polynomial solving: exact affine nonnegativity certificates. Such a service can exercise the reification, origin, guard, and evidence boundaries without pretending to solve all inequalities.

Let an affine row be a vector $r = (r_0, r_1, \dots, r_n) \in \mathbb{Q}^{n+1}$ with denotation

$$\llbracket r \rrbracket_v = r_0 + \sum_{j=1}^n r_j v_j \in \mathbb{R}, \quad v \in \mathbb{R}^n,$$

where rational coefficients are embedded into the reals. A request fixes hypothesis rows $r_1, \dots, r_m$ and a target row t . Its semantic specification is

$$\forall v \in \mathbb{R}^n, \quad (\forall i \leq m, 0 \leq \llbracket r_i \rrbracket_v) \longrightarrow 0 \leq \llbracket t \rrbracket_v. \quad (8)$$

Indices in an implementation should be finite indices with their bounds, not informal integers tested by truncating a list.

A certificate is a vector $\lambda \in \mathbb{Q}^m$ satisfying

$$\lambda_i \geq 0 \quad (1 \leq i \leq m), \quad t = \sum_{i=1}^m \lambda_i r_i \quad \text{as coefficient vectors.} \quad (9)$$

The checker tests the exact vector length, nonnegativity of the exact rational multipliers, and coefficient equality. No floating-point tolerance is involved.

Theorem 13.1 (Soundness of the affine certificate format). *Every certificate satisfying (9) establishes (8).*

Proof. Finite distributivity of real addition and multiplication, together with preservation of rational addition and multiplication under the canonical embedding, gives

$$\llbracket \sum_i \lambda_i r_i \rrbracket_v = \sum_i (\lambda_i : \mathbb{R}) \llbracket r_i \rrbracket_v.$$

This identity follows by expanding the constant coefficient and each variable coefficient, interchanging the two finite sums, and collecting terms. If every hypothesis denotation is nonnegative and each λ_i is nonnegative, each product on the right is nonnegative. Their finite sum is nonnegative. Coefficient equality in (9) identifies this sum with $\llbracket t \rrbracket_v$, proving the implication for the arbitrary valuation v . $\square$

This is a universal denotational argument, not a test at selected valuations. It is also deliberately not a completeness theorem for the checker. For example, a positive constant target may require a supplied trivial row $1 \geq 0$ for this exact format to certify it. Rejecting a proposed certificate does not refute the requested inequality.

13.2 The reification bridge is part of the theorem application

An expression reifier maps constants, fixed atoms, addition, subtraction, and rational scaling to coefficient vectors. Its correctness property is

$$\llbracket \text{reify}(e) \rrbracket_v = \text{eval}(e, v).$$

It is proved by structural induction: constants and atoms are immediate, addition uses coefficientwise addition, and scaling uses distributivity. A concrete Lean application additionally binds each atom to its exact elaborated real term and proves that the original expression is the corresponding evaluation. This binding is critical. The terms $u(2x + 1)$ and $u(2x - 1)$ are two atoms, not one atom because both are printed with the name u .

For arbitrary real expressions outside the affine grammar, the system can use a fixed expression as an opaque atom, provided it does not invent an equality between distinct atoms. A product of two varying atoms is not affine. Either an appropriate larger certificate service is selected, or the request remains unsupported. Approximate nonlinear simplification must not be hidden inside an affine reifier.

The Lean acceptance path therefore needs three proved facts: correctness of the reifier, soundness of the checker, and evidence that this particular check evaluates to acceptance. The paper proofs above specify the first two mathematical arguments. The Python companion executes a reference form of the third, but does not turn it into a Lean proof term. Porting all three to Lean is the first implementation milestone.

13.3 Certificates for the worked bounds

For a single value $a \in [0, 1]$, use rows a and $1 - a$. The target $2 - 2a$ has certificate $(0, 2)$ and $2a$ has certificate $(2, 0)$. For $a, b \in [0, 1]$, use rows

$$a, \quad 1 - a, \quad b, \quad 1 - b.$$

The upper-bound target $2 - 2(a - b)$ has certificate $(0, 2, 2, 0)$; the lower-bound target $2 + 2(a - b)$ has certificate $(2, 0, 0, 2)$. These are the exact identities in the proof of Theorem 10.1.

The interpretation back into the original derivative claim still uses the derivative law and the theorem characterizing an absolute-value bound by its two signed inequalities. Neither of these is replaced by an arithmetic success flag.

13.4 General symbolic computation fits the same boundary

Larger services can return other result types. Polynomial normalization returns an identity in a specified ring. An ideal-membership certificate can provide explicit coefficients for a combination of hypotheses. A root isolator returns a selected root's defining and isolating properties. A formal-series procedure returns a jet at a stated order, linked to a selected series by a proved relation.

These interfaces inherit the repaired round-two discipline [1, 2]. In particular, the residual-to-jet bridge needs a selected root, the appropriate constant term, and an invertible linearization under the theorem's assumptions. A nonzero derivative is not an invertible one over an arbitrary ring. Formal agreement to order N is not analytic convergence; differentiating a jet generally reduces its available order. Exact rational cancellation retains its domain or its total-operation case split.

The recommended default for imported Lean mathematics is to preserve Lean's existing operations, including their totalized cases. An explicitly partial expression package is permitted, but its domain must be part of its visible type. The frontend must not silently change a total expression into a partial one to make familiar simplifications valid.

13.5 A verified algorithm is not a trusted execution channel

Two service styles are legitimate. A verified implementation can be evaluated through a checked computational path. Alternatively, an untrusted producer can supply a certificate to a small verified checker. In either case, the proof concerns the original expression through the reification theorem, not just the producer's private data format.

A strict export policy should prefer ordinary kernel-checkable evidence. Any accelerated path with additional trust requirements must report its actual transitive axiom dependencies in the pinned environment, rather than rely on a remembered name for a computation axiom. The previous reports' timing probes are useful motivation for experiments but establish no threshold for this new workload [1, 2]. No such threshold is asserted here.

14 Leant integration: synthesize behavior, not just inhabitants

14.1 The contract is supplied before the candidate is accepted

A behavioral synthesis request has the dependent form

$$\sum_{f:A \rightarrow B} B_f(f),$$

where B_f is a fixed Lean predicate on candidates. The carrier type guides structural synthesis; the predicate states the required mathematics. A candidate may be rejected because the requested proof cannot be obtained, without concluding that the candidate is mathematically incorrect or that no solution exists.

The provider pipeline should keep four boundaries visible: a structurally generated candidate, acceptance of an exact rendered spelling by a callback, selection under a declared behavioral policy, and a checked Lean proof of the requested behavior. Leant’s current interfaces already distinguish the first three and explicitly decline to treat them as the fourth [10, 11, 12].

A proposed Lean-backed law registry would associate an exact provider term f with an exact theorem $p : B_f(f)$. The registry would verify that theorem in Lean before allowing its law to discharge a universal behavioral obligation. The existing passive transfer laws can remain useful for ranking and filtering; their successful origin binding must not be advertised as proving the mathematical law itself.

14.2 Length contracts are useful, but not enough for every request

A length law can eliminate obvious bad candidates. For example, the empty-list function usually fails a length-preservation request. But with a fixed element a_0 , the function

$$f(xs) = \text{replicate}(|xs|, a_0)$$

preserves length while generally changing every element. Thus a sorting specification needs a permutation or multiset-preservation property as well as sortedness under a selected order. Length preservation is one useful qualifier, not an implicit synonym for semantic correctness.

A finite test bank also cannot establish an unrestricted behavior law. Given any tested length cutoff N , a function that returns xs when $|xs| \leq N$ and the empty list otherwise passes all identity tests below the cutoff and fails the universal identity specification. There is an important exception: a proved exhaustive enumeration of a finite domain, combined with a sound checker, can establish a universal proposition over that domain. What matters is the coverage theorem, not whether the evidence was obtained by executing tests.

14.3 Where structural synthesis could genuinely help these examples

The Fabius example generates small proof-composition tasks: instantiate a universal range law; split conjunctions; feed the selected bounds and an arithmetic certificate to an absolute-value theorem; then feed a universal derivative bound to the Lipschitz theorem. The Frey example generates compositions of divisibility-preservation, subtraction, reflection, and gcd characterizations. Guarded computations generate implications that require combining local premises, splitting a conjunction of guards, or closing a case distinction.

These are concrete populations of obligations for a Leant adapter. Many may already be closed by one theorem application, `exact`, `aesop`, or ordinary elaboration. A fair evaluation must report that fact rather than credit Leant for every successful proof in a pipeline that happens to invoke it. The already short Frey contradiction is a particularly useful negative control: extra synthesis infrastructure may add cost without improving the source.

A practical request representation should preserve both the propositional skeleton used for structural search and the exact Lean expressions to which its atoms refer. A proof of the abstract skeleton transfers only through a proved interpretation of the skeleton into the original proposition. An abstraction’s non-derivability does not supply a proof of the original proposition’s negation [1, 2].

14.4 Exact edits and retained authority

For tactic suggestions, the text displayed to the author must be replayed as the exact proposed edit in the originating proof state. A probe of a nearby tactic, an earlier spelling, or a different local context does not validate that edit. A decreased goal count establishes local progress, not completion of the theorem.

The retained record should contain the exact accepted term or edit, the original target and context, the interpretation bridge when an abstraction was used, the resulting proof evidence, and the applicable policy. A fresh request can reuse evidence through a checked adapter, but not merely by comparing names or copying a receipt. These requirements extend the existing explicit boundaries rather than treating Leant as an untrusted black box with a magical “verified” output.

15 Should Lean’s kernel type system change?

15.1 The recommended answer: not for the first implementation

All complete rules in this article elaborate to existing dependent functions, propositions, equalities, subtypes, and structures. The property index is an elaboration aid. Behavioral weakening is an implication proof. Missing index equalities are solved by producing equality evidence and inserting explicit transport. None of this requires a new axiom or an unrestricted conversion rule.

An important example is a dependent type indexed by $m + n$ where an interface expects $n + m$. A property-aware elaborator may prove the index equality and insert an explicit cast. That is different from changing definitional equality so that every arithmetically equal index is automatically convertible. The cast can be hidden in the mathematical display while remaining present in the checked term.

The first implementation should therefore test whether the source-language extension alone captures the benefits. A successful library and elaborator plugin are not a compromised outcome. They preserve interoperability and allow the unchanged Lean kernel to check exports.

15.2 A narrowly stated kernel research option

A future experiment could investigate a primitive ghost-refinement or a restricted coercive-subtyping mechanism with a proved translation to the original core. Candidate benefits would include less elaboration overhead, simpler generated transports, or more predictable normalization for specific data families. Each benefit would need a workload on which the elaborator-only implementation actually loses.

The required metatheory would include well-formedness and substitution, preservation of typing under reduction, proof or coercion coherence at the chosen boundary, and a precise account of the conversion procedure. A translation must preserve the meaning of data and the exact types of exported statements. Mechanized results for another dependent calculus, such as the work on definitional functoriality, are relevant precedents but not proofs about an extension to Lean [24]. Lean4Lean provides another relevant line of work on specifying and checking Lean’s core; it is not evidence that an arbitrary proposed kernel change is safe [25].

Option	Possible benefit	Decision here
Proof-explicit refinement elaboration	Better property inference with ordinary proof terms	Recommended first implementation
Automatic propositional index transport	Fewer manual dependent casts	Elaborator service; retain equality evidence
Restricted definitional functor/coercion laws	More convenient normalization for selected type formers	Research only, with separate metatheory and measured need
Unrestricted equality reflection	Any proved equality becomes conversion	Not part of the proposal; moves arbitrary proof obligations into conversion
CAS evaluation inside definitional equality	Apparently effortless algebraic conversion	Rejected as a default; use certificates and explicit equality proofs instead
Arbitrary implicit refinement entailment	Very expressive apparent subtyping	Expressible as goals, not a promised terminating inference procedure

15.3 Proof irrelevance is not mathematical extensionality for free

Proof irrelevance identifies proofs of one proposition. It does not equate two rings on a carrier, choose an isomorphism, turn a pointwise equality into neighborhood equality, or identify two algebraic roots. Function extensionality can prove equality from equality at every input, but those universally quantified premises still need proofs. A quotient respects its designated equivalence relation through the appropriate elimination theorem; it does not authorize arbitrary representative changes under operations that have not been shown well defined [14, 17].

The type-level convenience should therefore be *explicit evidence inserted automatically*, not additional mathematics silently treated as computation. This keeps a failed inference intelligible: the missing item is a theorem or a premise, rather than a mysterious change in the kernel's notion of sameness.

16 A document view that cannot overstate its evidence

16.1 One semantic source for code and mathematical presentation

Trellis should retain a typed argument graph from which it produces both ordinary Lean exports and a mathematical document. A claim node records its exact proposition, data origins, context, method role when specified, and evidence dependencies. An observation node additionally records its result relation: equality, a bound, an enclosure, a jet, a witness, or a complete set.

The reader's default view must expose every distinction that changes the mathematical assertion: domains, branches, precision, completeness, unresolved conditions, and relevant trust policy. Routine proofs of already visible conditions may be folded. For example, a proof that $2 > 0$ can disappear; the fact that a formula requires $x \neq 0$ cannot disappear from an unrestricted result. An interval enclosure should print its exact bounds even when a rounded decimal is shown for convenience.

The source and display should not use independent parsers that merely agree on names. The same elaborated objects and propositions must drive both. Even then, no formal mechanism can prove that an informal reader intended the same mathematics. A readable statement seal and semantic-reading tests remain necessary.

16.2 A stated reason is a constraint, not decorative prose

The phrase `by derivative_bound` selects a theorem role and a derivation node with the required premises. The accepted local claim must be obtained through that registered rule, or the interface must report that a different method was used. Merely inserting a reference to the named theorem somewhere in an otherwise unrelated proof does not validate the narrated step.

There is a subtler limit. A theorem can formally accept a premise it does not use. A source may supply that premise, and an application can be well typed, without the proof term depending on it computationally or logically. The FLT proof-path discussion makes this distinction concrete [8]. The document view should distinguish *required by the public theorem interface*, *supplied to this application*, and *present in the retained proof dependencies*. It should not make an unsupported claim that every named premise was mathematically indispensable.

16.3 Editing, cases, and reuse

An edit that changes only a proof can preserve the statement seal. An edit that changes a carrier, a structure, a witness, or an observation kind requires a meaning-change notice. Acknowledging that notice does not discharge a new proof obligation.

At a case split, a fact proved in both branches can be joined by ordinary case elimination. It is not imported into the parent merely because one branch’s qualifier cache contains it. At an import or toolchain upgrade, retained evidence is rechecked or adapted to the new environment, with a new receipt. Old evidence may remain useful, but an obsolete digest or policy label never creates new proof authority.

General reuse has a theorem-shaped interface. The new context must establish the old assumptions under a checked substitution; objects must correspond; the old guarantee must imply the requested one; and the evidence must meet the destination policy. The correct variances follow: a larger validity domain answers a smaller-domain request; a more precise jet answers a lower-precision request; a narrower enclosure answers a wider one. An assumption that is stronger than what the new context provides does not transfer automatically [1, 2].

17 Implementation plan and the evidence actually supplied

17.1 A minimal vertical implementation, not a maximal language

The first implementation should consist of four small components. A library declares the refinement and behavioral contracts as ordinary Lean propositions. A role registry associates selected theorem constants with checked input/output roles and versions. A demand elaborator instantiates a finite qualifier graph and assembles proofs. A certificate adapter implements the affine format of Section 13, including reification and checked execution.

Initially the interface can be ordinary Lean with annotations on theorems and explicit API calls. A hypothetical `@[trellis_rule]` annotation would index a theorem; it would not trust its English documentation or infer semantic argument roles solely from binder names. Each registration must be checked against its actual theorem type and the particular matching discipline it supports.

Only after that path works should the proposed surface syntax be added. The first end-to-end acceptance criterion is modest but real: a property demand about an existing Lean expression, a generated exact certificate, a proof at the original goal with the specified axiom policy, and rejection of a mutated certificate or missing premise. The Fabius upper-bound fragment is small enough to serve that role without requiring a whole symbolic-computation platform.

17.2 What the companion implements

The supplied `companions/trellis_reference.py` uses only Python’s standard library. It provides an affine expression reifier, an exact-rational multiplier-certificate checker, origin-bound requests, and a deliberately restricted parent-to-child scope-weakening operation. Request digests include the target expression, hypothesis expressions, atom handles, environment and structure identifiers, revision, policy identifier, context, and result kind.

The checker accepts only the exact affine nonnegativity relation it implements. It rejects wrong arity instead of truncating a zipped list, rejects negative multipliers, distinguishes atom applications, rejects floating-point coefficients, and checks the exact coefficient identity. An explicit scope-weakening operation verifies the source certificate, checks that only a valid context-prefix extension occurred, and rechecks the certificate against the destination request.

These are *reference semantics*. String identifiers stand in for externally supplied opaque Lean origins; the script does not obtain or authenticate Lean expressions. Its scope-prefix rule does not implement arbitrary dependent-telescope transport. A hash is used to detect mismatches, not to prove identity of mathematical objects. The script neither calls Lean nor invokes a Lean kernel, and its Python implementation has not been formally verified.

17.3 Executed checks and their limits

The supplied test run records **31 passing reference tests: 9 acceptance cases and 22 rejection cases**. The JSON records each expectation, observed result, and diagnostic. It includes the four concrete derivative-bound certificates, exact rational multipliers, an empty zero certificate, corrupted coefficients, certificate-length errors, changed origins and policies, unsupported guarantee kinds, distinct atom applications, and scope-escape attempts.

A rejection case is not necessarily a false mathematical claim. It may be a true claim with a malformed or unsupported certificate, or a valid old result that cannot be inserted into a changed request without revalidation. This distinction is intentional. The count is a reproducible engineering observation about the companion, not a language benchmark or a claim that the 51 families from the prior reports have been implemented.

No Lean compiler or kernel was available in the working environment used for this article. Accordingly, no Lean compilation, repository rebuild, axiom audit, or claimed proof-compression experiment is attributed to this work. The paper’s core translation theorem is a mathematical proof for the specified rules, not a verified compiler result. The PDF was compiled and inspected independently of these logical implementation claims.

17.4 Additional paired tests for a Lean implementation

Boundary	Positive case	Negative neighbor
Prime refinement	Prime $p \geq 5$ implies oddness	Prime $p = 2$ must not acquire oddness
Behavioral reflection	$d^p \mid e^p$, $p > 0$, over integers	Zeroth powers, or only $d \mid e^p$
Derivative law	HasDerivAt evidence plus value identity	A totalized derivative expression without existence evidence
Local transport	Equality on an open neighborhood of the point	Equality only at the point

Boundary	Positive case	Negative neighbor
Fabius upper bound	Range and derivative laws imply 2-Lipschitz	Upper bound alone claimed to prove leastness
Small-error limit	Error tends to zero on a specified right domain	A finite list of small numerical errors
Exact quotient cast	Divisibility proves integer division exact	Cast of 1 div 4 identified with 1/4
Function synthesis	Sortedness and permutation under a fixed order	Length preservation substituted for permutation
Structure stability	Same operation dictionaries, new proof evidence	Changed multiplication or scalar action under unchanged notation
Context discipline	A closed conditional theorem or valid weakening	A sibling branch's guard retrieved by printed name
Rule closure	Acyclic evidence assembled from seeded facts	A guard justified only by the result it guards
Observation kind	Certified finite jet returned as that jet	Same certificate displayed as an infinite analytic identity

These are proposed integration tests, not additional tests executed by the Python companion. Most reuse the earlier reports' mutation discipline; the new examples specify where that discipline meets the proposed property typing.

18 Evaluation and stopping rules

18.1 Treatments that isolate the source of a gain

The decisive experiment should use nested treatments on the same mathematical tasks and library revisions. The first is well-written ordinary Lean. The second adds the same mathematical wrappers and certificate services. The third adds scoped property inference and retained structure selections while keeping ordinary Lean syntax. The fourth adds behavioral synthesis. Only the final treatment adds the Trellis mathematical surface.

This separates library improvement from inference improvement, inference from synthesis, and both from notation. Otherwise a short new-language example can win merely because its baseline was denied the theorem wrapper that does most of the work. A variant that keeps behavioral synthesis but removes automatic qualifier closure can further test whether the layers complement each other.

18.2 Metrics must include mathematics and maintenance

Authoring measurements should include active editing time, failed attempts, manual statement repairs, and infrastructure introduced for the task family. Reader measurements should ask for the exact theorem's objects, quantifiers, hypotheses, domain, and guarantee kind before showing expanded evidence. Acceptance rates for misleading compact statements are as important as reading speed.

Engineering measurements should separate discovery from replay, and elaboration from kernel checking. Record tail latency, memory, proof-term size, rule-generation size, certificate size, and import sensitivity. For source compression, count semantic declarations and proof obligations as

well as visible tokens; separately account for generated preambles. For maintenance, change a theorem signature, an instance selection, a domain premise, or an index convention and measure the repairs required.

No numerical gain is predicted as a fact. Pilot measurements can inform sample sizes and resource limits; evaluation thresholds should then be fixed before opening the sealed test tasks.

18.3 Development examples are not held out

The Fabius, stationary-phase, and Frey examples are now part of the design. They cannot supply an unbiased estimate of generalization. New evaluation tasks must be selected separately, including checks against aliases, direct corollaries, and close dependency leakage. The report-to-report fourth-file rule is useful for broadening design experience, but a new filename alone is not a sealed benchmark [1, 2].

Use more than one mathematical domain and more than one proof style. Include already concise proofs, problems where structure ambiguity matters, proofs whose hard part is a genuinely new lemma, and tasks where finite qualifier inference predictably reaches its boundary. A system should not be rewarded for refusing difficult tasks while presenting only favorable examples.

18.4 When to stop adding language

The property layer succeeds if it provides reliable, inspectable inference and improves author or reader outcomes beyond the shared-library baseline. The surface syntax succeeds only if it adds value beyond that layer. A kernel change succeeds only if it improves a measured bottleneck that the conservative implementation cannot address adequately, while meeting the required metatheory and compatibility constraints.

If ordinary Lean with the same contracts performs as well, keep the contracts and stop building the language. If an explicit structure manifest helps reproducibility but new syntax does not, ship the manifest tooling. If Leant contributes little beyond direct theorem application on the measured obligations, do not put it on every proof path. A library-only outcome is a successful finding, not an embarrassment.

19 Answers to the third-round design questions

The two syntheses pose overlapping sets of questions. Rather than repeat all earlier tables, the following gives operational decisions for this proposal [1, 2].

What is the smallest useful first slice? One affine-bound request about existing Lean expressions, with a proved reifier, a proved checker, a checked execution, and a retained exact-target proof. Add qualifier propagation for that request before adding another certificate language.

What is new state, rather than ordinary Lean context? The role-indexed evidence lookup, frozen structure manifest, source-to-proof graph, and replay/migration receipts. The actual variables, hypotheses, theorem terms, and local instances remain ordinary Lean state. Each extra index must justify its maintenance cost.

Partial or total operations? Preserve the semantics of the imported Lean terms by default. Offer a separate, explicit partial-expression interface with a visible domain. Never change between them as an invisible proof-search convenience.

What may automation choose silently? Proofs of a fixed proposition, routine theorem instantiations with already determined data, and certificates whose exact guarantees are fixed. It

may not silently choose a new mathematical object, domain, root, operation, or stronger theorem hypothesis. Declared construction holes are an explicit exception with a fixed specification.

How are refinements inferred? By bounded generation of a grounded qualifier graph and proof-producing closure; broader search remains a distinct provider. Unknown is not false. There is no general promise of principal mathematical types.

Where does a bounded Gröbner tactic belong? As a provider with its actual coefficient-domain, resource, and failure contract. Trellis does not claim completeness from its name and does not need it for the first affine slice. Whether it can emit the desired retained certificates is an implementation question left open here.

When is accelerated execution worth its additional trust? Only after paired measurements on the actual checker data and explicit revalidation of the destination axiom policy. No universal size threshold follows from the previous round’s particular multiplication probes.

What does Leant contribute specifically? Candidate composition for the explicitly identified implication, conjunction, instantiation, and behavioral-adaptor obligations. Its callback and behavioral-selection receipts remain distinct from the final Lean proof. A direct theorem-application baseline is mandatory.

What must be visible? Any unresolved hypothesis or accepted distinction affecting objects, quantifiers, domain, branch, precision, completeness, or policy. The automatic proof of a visible routine condition may be folded. A stated method is either respected by the argument graph or reported as replaced.

How does evidence survive change? By exact replay in the pinned environment or a checked adaptor into the new one, with a fresh receipt. Reuse is governed by implication and context transport, not by a cache hit. Data-changing edits require a new statement or object seal.

Which kernel extension is justified now? None is required by the presented calculus. Restricted coercion or normalization changes are a separate research program, contingent on measured need and a proved relationship to the original core.

What has this article actually established? A specified conservative refinement discipline; mathematical proofs of its small-core translation and finite-closure properties; complete proofs of the worked analytic and algebraic consequences; an affine-certificate soundness argument; and a tested executable reference model. It has not established a compiled Lean frontend, a verified compiler, broad automated synthesis success, or measured usability gains.

20 Conclusion

The strongest reason to extend a proof language’s type interface is not that Lean cannot express positive numbers, smooth functions, or behavioral specifications. It can. The reason is that those properties should participate in elaboration in a way that is predictable, scoped, compositional, and visible to a mathematician.

Trellis proposes that an object’s raw term remain stable while its proved properties become available through a demand-directed evidence layer. Mathematical structures are selected as data; mathematical properties are inferred as proofs. Function contracts record actual behavior, including preservation and reflection of specified relations. Computer algebra returns a proof-relevant result relation rather than an expression whose guarantee the reader must guess. Leant supplies candidates and proof compositions without being asked to turn its acceptance receipts into authority they do not claim.

The new examples make the intended boundary concrete. A range law and a derivative law can automatically yield a Lipschitz bound, but optimality needs its own witness. A quotient derivative needs analytic guards even when its numerator identity is easy. A vanishing-error

contract can replace repeated epsilon arithmetic, but not a missing convergence proof. Frey coefficients can cross from integers to rationals when exact divisibility is proved, not merely because their formulas look the same.

The proposed first result is therefore modest in syntax and strict in semantics: one small, end-to-end Lean evidence service; a finite property-inference layer over it; and a comparison against ordinary Lean equipped with precisely the same mathematics. The language should grow only where that comparison shows that it helps.

A A minimal grammar and export discipline

The grammar below is a design sketch. It is not an implemented parser, and theorem-role names stand for checked registrations rather than English inference rules.

```

declaration := variable name : type [with property_row]
              | let name [: type with property_row] := term
              | construct name : type satisfying proposition
              | refine term with property_row by evidence

argument     := claim proposition : steps
              | for arbitrary binder : argument
              | assume proposition : argument
              | cases proposition : branches
              | conclude by role [using named_evidence]

observation  := compute request returning result_kind
              [on domain] [under explicit_conditions]

property_row := predicate | property_row and predicate
result_kind  := equality | inequality | enclosure
              | jet precision | witness | complete_set
              | registered_relation

```

The result-kind alternatives are an initial vocabulary, not a closed universe. Each has a precise Lean proposition and may carry dependent parameters. A registered relation cannot declare itself stronger than another by assigning a numeric rank; a bridge theorem is required.

A complete export performs the following logical work. It checks the sealed statement; resolves every claimed formal assertion; checks all proof terms and certificate executions against the exact targets; validates the selected axiom policy; and emits the ordinary Lean declarations plus the evidence graph needed to interpret the mathematical view. Search policy and presentation metadata do not become axioms in those declarations.

A standalone saved proof need not retain the entire unsuccessful search history. It must retain enough accepted evidence and environment information for replay. Optional discovery provenance can support debugging or evaluation, but replay correctness must not depend on contacting a language model, a CAS service, or a mutable online index again.

B Reproducing the companion checks

From the archive’s top-level directory, run:

```

python3 companions/trellis_reference.py \
--output companions/results.json

```

The supplied run reports:

```
31 tests passed (9 positive, 22 negative).
No Lean compiler or kernel was run.
```

The reference checker uses exact `Fraction` arithmetic. Its rejection of floats is intentional. The digest check is a protocol safeguard, not a theorem about a mathematical statement. The restricted scope operation only accepts context-prefix extensions with the other semantic fields unchanged; a full Lean implementation needs dependent substitutions and proof-term checking. To rebuild this article, use the supplied `build.sh`, or run `latexmk -pdf trellis.tex`. All bibliography entries are embedded in the TeX source. The compiled PDF, source, companion, test results, source register, and README are included in the archive. No downloaded font files or external document assets are needed.

C Source register and provenance

The source register identifies what was read, not what was independently rebuilt. The `ProveIt` regularity file and Frey-package definition file were read in full. The stationary-phase files were inspected in the portions supplying the definitions, derivative contracts, quotient calculation, bounds, and small-error helper discussed here. The generated final Frey proof was inspected for its environment-management preamble and mathematical conclusion. Both requested round-two syntheses were read through their discussions, corrections, appendices, and source maps.

The directly inspected Leant interfaces expose verification receipts, behavioral-selection sealing, and Length contracts. No claim is made to have reviewed the entire synthesis engine or all of its provider implementations. The earlier syntheses' deeper Leant assessments are attributed to those reports where used.

Exact repository revisions:

```
Brainstorm:
  b052ec011beb96a7c48df2a9988823f0f779b445
ProveIt:
  24ce8bd743eaab64a91ce90725ea00f498d319d2
anthropics/fermats-last-theorem:
  aa2d8b34692b16c70f699536de0d8e75b9a3e9ef
Leant:
  3a40904be8a410d832d9ab6900b3d3e7b425eccb
```

The source Frey package credits the Imperial College London FLT formalization and its named contributors. This article proposes interfaces around that mathematics; it does not claim authorship of the underlying package or the FLT proof. The Fabius and stationary-phase results are likewise credited to the inspected `ProveIt` development. The vanishing-error generalization and the language metatheory are derived explicitly in the article; no novelty claim is made for their underlying classical mathematical ideas.

References

- [1] Codex. *Mathematical Objects, Certified Computation: A Unified Evaluation of Nine Proof-Language Proposals*. Round-two synthesis, revised 6 September 2026. Brainstorm revision `b052ec011beb823f0f779b445/docs/round-2/synthesis/unified-report.tex`.
<https://github.com/VladimirReshetnikov/Brainstorm/blob/b052ec011beb96a7c48df2a9988823f0f779b445/docs/round-2/synthesis/unified-report.tex>

- [2] Claude. *Nine Workbenches: Certified Computation in a Mathematician’s Proof Language over Lean*. Round-two synthesis, revised 6 September 2026. Same Brainstorm revision.
https://github.com/VladimirReshetnikov/Brainstorm/blob/b052ec011beb96a7c48df2a9988823f0f779b445/docs/round-2/unified_report/unified_report.tex.
- [3] V. Reshetnikov and contributors. *ProveIt, Regularity.lean* (Fabius and Rvachev regularity). Revision 24ce8bd743ea.
<https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/Analysis/FabiusFunction/Lean/FabiusFunction/Regularity.lean>.
- [4] V. Reshetnikov and contributors. *ProveIt, Stationary phase, part 1: the Taylor bounds, SP1Taylor.lean*. Same revision.
<https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/NumberTheory/IntegerPoints/Lean/IntegerPoints/SP1Taylor.lean>.
- [5] V. Reshetnikov and contributors. *ProveIt, Stationary phase, part 2, SP2Parts.lean*. Same revision.
<https://github.com/VladimirReshetnikov/ProveIt/blob/24ce8bd743eaab64a91ce90725ea00f498d319d2/NumberTheory/IntegerPoints/Lean/IntegerPoints/SP2Parts.lean>.
- [6] Anthropic and contributors. Fermat’s Last Theorem repository, *Def_FLTPrelim_FreyPackage.lean*. The file credits K. Buzzard, R. Van de Velde, and P. Monticone and its Imperial FLT provenance. Revision aa2d8b34692b.
https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/Definitions/Def_FLTPrelim_FreyPackage.lean.
- [7] Anthropic and contributors. *S_FreyPackage_no_frey_package.lean*, same revision.
https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/P2M/Sol/S_FreyPackage_no_frey_package.lean.
- [8] Anthropic and contributors. *The proof path, PROOF-PATH.md*, same revision.
<https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/PROOF-PATH.md>.
- [9] Anthropic and contributors. Fermat’s Last Theorem repository, *README.md*, same revision. Repository build and validation reports; not independently rerun here.
<https://github.com/anthropics/fermats-last-theorem/blob/aa2d8b34692b16c70f699536de0d8e75b9a3e9ef/README.md>.
- [10] V. Reshetnikov and contributors. *Leant, src/Leant/Synth/Verification.hs*. Revision 3a40904be8a4.
<https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Verification.hs>.
- [11] V. Reshetnikov and contributors. *Leant, src/Leant/Synth/BehavioralSelection.hs*. Same revision.
<https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/BehavioralSelection.hs>.
- [12] V. Reshetnikov and contributors. *Leant, src/Leant/Synth/Length/Contract.hs*. Same revision.
<https://github.com/VladimirReshetnikov/Leant/blob/3a40904be8a410d832d9ab6900b3d3e7b425eccb/src/Leant/Synth/Length/Contract.hs>.
- [13] Lean team. *Subtypes*. Lean Language Reference, accessed 6 September 2026.
<https://lean-lang.org/doc/reference/latest/Basic-Types/Subtypes/>.
- [14] Lean team. *The Type System*. Lean Language Reference, accessed 6 September 2026.
<https://lean-lang.org/doc/reference/latest/The-Type-System/>.
- [15] Lean team. *Type Classes*. Lean Language Reference, accessed 6 September 2026.
<https://lean-lang.org/doc/reference/latest/Type-Classes/>.
- [16] Lean team. *Instance Synthesis*. Lean Language Reference, accessed 6 September 2026.
<https://lean-lang.org/doc/reference/latest/Type-Classes/Instance-Synthesis/>.

- [17] Lean team. *Quotients*. Lean Language Reference, accessed 6 September 2026.
<https://lean-lang.org/doc/reference/latest/The-Type-System/Quotients/>.
- [18] Lean team. *Validating a Lean Proof*. Lean Language Reference, accessed 6 September 2026.
<https://lean-lang.org/doc/reference/latest/ValidatingProofs/>.
- [19] Lean team. *Tutorials*, including *Verifying Imperative Programs Using mvcgen*. Accessed 6 September 2026.
<https://lean-lang.org/doc/tutorials/latest/>.
- [20] Lean team. *Lean 4.33.0 release notes*. 10 August 2026, accessed 6 September 2026.
<https://lean-lang.org/doc/reference/latest/releases/v4.33.0/>.
- [21] P. M. Rondon, M. Kawaguchi, and R. Jhala. *Liquid Types*. PLDI 2008. Authors’ publication page.
https://goto.ucsd.edu/~rjhala/papers/liquid_types.html.
- [22] F* team. *Proof-Oriented Programming in F*: Getting off the Ground*; and the project overview. Accessed 6 September 2026.
https://fstar-lang.org/tutorial/book/part1/part1_getting_off_the_ground.html;
<https://fstar-lang.org/>.
- [23] N. Polikarpova, I. Kuraj, and A. Solar-Lezama. *Program Synthesis from Polymorphic Refinement Types*. PLDI 2016; arXiv:1510.08419v3.
<https://arxiv.org/abs/1510.08419>.
- [24] T. Laurent, M. Lennon-Bertrand, and K. Maillard. *Definitional Functoriality for Dependent (Sub)Types*. ESOP 2024; extended version arXiv:2310.14929.
<https://arxiv.org/abs/2310.14929>.
- [25] M. Carneiro. *Lean4Lean: Towards a Verified Typechecker for Lean, in Lean*. arXiv:2403.14064, 2024.
<https://arxiv.org/abs/2403.14064>.